

agent-hands — Engineering Reference

Code, frameworks, and the requirement map · prepared for the
interface.ai build session

Ashish Kosana

19 August 2026

Contents

0 • How to use this document	4
1 • Repo at a glance	5
2 • The one-minute pitch	6
The five invariants	6
The two-path architecture	7
3 • Frameworks, and the case for each	8
Defending each choice	8
4 • Requirement map	10
§3.1 — Goal-driven agent loop	10
§3.2 — Structured artifact	11
§3.3 — Deterministic replay	11
§3.4 — Safety & policy	11
§3.5 — Evidence / observability	12
§3.6 — Escalation & handoff	12
§3.7 — Heterogeneity & multi-tenant (design)	12
5 • The artifact, annotated	14
The load-time integrity validators	14
Why this shape and not a step list	15
6 • Determinism	16
The locator ladder	16
Geometric rungs — reading table-soup	17
The timeout model — two levels	17
The freshness rule — two halves	17
No double fire	18
Values are parsed, never string-compared	18
Measured results	19
The zero-LLM proof	20
7 • Error taxonomy	21
The five-way result contract	21
Three classes of runtime state	21
Recognizer provenance and arming	22
8 • Escalation & handoff	23
The control-transfer model	23
The state machine	23
Taking control of the <i>same</i> live session	24
Bounded, not open-ended	24
What is mocked, stated plainly	25
9 • The discovery seam	26
Contract-first	26
Model proposes, recorder proves	26
One run per ending	27

10 • Evidence inventory	28
11 • Built vs designed-not-built	29
Built and tested	29
Designed, not built	29
12 • Known discrepancies	30
13 • Likely hard questions	31
14 • Commands	33
Editing gotchas	34

0 · How to use this document

This is a defense reference, not a tutorial. Everything here is traceable to a file and line in the repo. It is organised so that any question of the form “*where is X and why did you do it that way?*” has one place to look.

The document assumes the assessment brief’s section numbering (§3.1 goal loop, §3.2 artifact, §3.3 replay, §3.4 safety, §3.5 evidence, §3.6 escalation, §3.7 heterogeneity).

Reading order if time is short: §2 (the one-minute pitch), §5 (the artifact), §6 (determinism), §12 (known gaps), §13 (likely hard questions).

1 · Repo at a glance

```
src/hands/      4,668 lines · 15 modules · the system
fixture/       487 lines · the "Fairview Teller" legacy target app
tests/        1,555 lines · 12 files
evals/         209 lines · offline eval runner -> evals/results.md
capabilities/   artifacts + discovery requests
evidence/      discovery + replay + escalation run records
docs/DESIGN.md full design incl. considered-and-rejected
docs/DEFENSE.md anticipated hard questions
REPORT.md      as-built summary (the seven required headings)
```

Module line counts:

MODULE	LINES	RESPONSIBILITY
replay.py	1009	the deterministic engine — the heart
artifact.py	527	the contract: schema + load-time integrity validators
planner.py	617	LLM observe -> decide -> act loop
discover.py	447	discovery orchestration, contract-first
surface.py	438	the perceive/act seam (only Playwright-aware module)
recorder.py	385	distillation: model proposes, recorder proves
escalation.py	327	control-token state machine + operator console
observe.py	197	what the planner sees; refs + pinned handles
conditions.py	191	sensors only — never act on the page
cli.py	140	replay / review / discover
llm.py	134	provider seam (any OpenAI-compatible endpoint)
values.py	111	placeholders, normalize, strict money parse
results.py	79	the 5-way result union
trace.py	57	append-only JSONL audit log

2 · The one-minute pitch

Record-once / replay-many computer-use automation for legacy UIs with no API.

An LLM works out a task inside a real UI **once** (*discovery*). That run is distilled into a typed, versioned **capability artifact** (JSON). Production invocations **replay** the artifact deterministically — **no model in the decision loop at replay**, proven by a hermetic test rather than asserted.

The artifact is a **contract first, a step list second**. `parameters`, `outputs`, and a closed set of `outcomes` live at the top level so a calling agent can invoke it blind; steps, locator ladders, and recognizers are implementation detail below that line.

The five invariants

Every design decision in the system serves one of these. If a change would violate one, that is the thing to stop and flag.

1. **Never guess.** Ambiguity at replay is a failure, not a heuristic. A rung matching more than one visible element is a hard failure, never “pick the first.”
2. **Wrong-but-confident is the worst outcome.** Uniqueness is not correctness. Every resolved element is fingerprint-verified; every extracted value is parse-verified against recorded expectations.
3. **A business outcome is an answer, not an error.** The result contract is five-way, and a caller can enumerate every code it may receive.
4. **Irreversible (`risky`) actions are never auto-retried and never replayed unattended without a human-signed, hash-bound review.**
5. **Replay sends nothing to any model, ever** — enforced by `tests/test_zero_llm.py` (fresh subprocess, keys stripped, non-loopback sockets blocked).

The two-path architecture

DISCOVERY (once, LLM, costs money)	REPLAY (many, no LLM, ~\$0, proven)
request.json (contract seed)	capability.json + params
planner: observe -> decide -> act (tool-forced, ref-grounded)	
recorder: distill & round-trip-verify	engine: requires -> per step:
every rung/anchor/marker live	pre -> ladder -> act(retry) -> post,
----->	recognizers polled throughout ->
capability artifact (JSON)	checkpoint(identity) -> outputs
	SUCCESS BUSINESS_OUTCOME FAILURE
	PRECONDITION_FAILED POLICY_VIOLATION

Shared: policy (network allowlist, risk gate) · trace (JSONL + masked evidence)
 escalation (pause -> human on the SAME live session -> resume)

3 · Frameworks, and the case for each

CHOICE	WHERE	WHY THIS ONE
Python 3.12+	<code>pyproject.toml</code>	Pydantic + Playwright + strict mypy is the best-typed stack for a schema-centric problem. <code>requires-python = ">=3.12"</code> for modern union syntax throughout.
Pydantic v2	<code>artifact.py</code> , <code>results.py</code>	The artifact is the deliverable, so the schema had to be executable, not documentation.
Playwright (sync API)	<code>surface.py</code> only	Chosen for the role engine . Sync API deliberately — it is thread-affine, which forces an honest single-owner control model in escalation.
Flask	<code>fixture/app.py</code>	The target app: server-rendered, zero JS, nested tables, no ids, no <code><label for></code> , a nav iframe.
<code>openai</code> SDK	<code>llm.py</code>	Used as a protocol client , not a vendor commitment. 134 lines wrapping any OpenAI-compatible endpoint.
stdlib <code>http.server</code>	<code>escalation.py:227</code>	The operator console. No web framework on purpose — it is the declared mock, and a dependency there would overstate what it is.
pytest / ruff / mypy -strict	12 test files	mypy strict over <code>src</code> , <code>fixture</code> , <code>tests</code> , and <code>evals</code> .

Defending each choice

Why Pydantic and not dataclasses or JSON Schema? The load-time validators are the product. Discriminated unions give genuinely closed sets — rung strategies, condition kinds, the five result shapes — and `extra="forbid"` makes a typo in an artifact fail at load rather than at 3am in production. All cross-field invariants live in `@model_validator`s in one file, so “what makes an artifact valid” has a single answer.

Why Playwright and not Selenium / a CUA SDK / screenshot+coordinates? The deciding factor was `get_by_role` — Playwright’s computed accessibility-tree query. That is the surface that also exists on desktop (UIA on Windows, AX on macOS), so the primary locator strategy ports. Screenshot+coordinates was rejected explicitly: coordinates are the least stable thing about a UI, and a coordinate replay cannot tell “the button moved” from “the button is gone.” There are no coordinates anywhere in the replay path.

Why the sync API when async is the default advice? Because it is thread-affine, and that constraint produced a better escalation design than async would have. The console cannot touch the browser even by accident; it can only post requests into a hub that the engine thread drains. Single ownership is structural rather than disciplined.

Why Groq / gpt-oss-120b? Because it does not matter, which is the honest answer. `llm.py` is a thin OpenAI-protocol client; provider, base URL, and model are config (`HANDS_MODEL` or `--model`). The default is a fast, cheap open model that supports forced tool use. Swapping providers is one config change, and the system's correctness properties do not depend on model quality — the recorder verifies everything the model proposes.

Why build a fixture instead of using a public demo site? Public demo sites are too clean. The interesting problems in this domain come from table-soup markup, missing label associations, frames, and runtime error states you can *trigger on demand*. The fixture has six injectable faults via `POST /__faults`, which is what makes the error taxonomy testable rather than aspirational.

4 · Requirement map

§3.1 — Goal-driven agent loop

`planner.py` (617) + `observe.py` (197) + `llm.py` (134).

The loop is **tool-forced and ref-grounded**:

- `tool_choice="required"`, `parallel_tool_calls=False`, `temperature=0.0` (`llm.py:65`). The model cannot emit prose and gets exactly one action per turn.
- Four tools only (`planner.py:52`): `act`, `done`, `report_outcome`, `stuck`. `report_outcome` existing at *discovery* time is the point — “no such member” is an answer even while learning.
- **Act-by-ref**: `act` takes a `ref` from the current observation (`observe.py:55`). The model cannot invent a selector. Each `ref` pins a real `ElementHandle`, so DOM churn between `observe` and `act` cannot swap the target.
- **Protocol safety** (`planner.py:304–334`): every proposed tool call is answered on the tool channel, even when rejected. Answering a rejected call with a user message 400s the next request. This was a real bug, found and fixed.
- Stop conditions: `done` / `report_outcome` / `stuck` / too many invalid proposals / step-budget exhaustion — with **one grace turn** (`planner.py:337–348`) so a flow that completes on its final budgeted step still gets to call `done`.

Keeping values out of the loop: parameters are `{param:name}` placeholders the model writes symbolically and the surface resolves at act time; the planner **rejects** a `type` whose literal text contains a concrete parameter value (`planner.py:458`). Password fields are masked before observation (`observe.py:158`).

On the brief’s “bias toward an approach that would still work without a clean DOM”: the observation is **frame text plus role-enumerated actionable nodes**, not HTML. There is no markup in the prompt at all. That is the same information a screen reader has.

§3.2 — Structured artifact

`artifact.py` (527). Shape:

```
schema_version, name, version, description, target
requires, parameters, outcomes, outputs      <- a calling agent reads only this
-----
regions, steps, conditions, checkpoint        <- implementation detail
allow_unbound_checkpoint, risk_review
```

Versioning is two-layer: `schema_version` pins **ladder semantics** (exact whitespace-normalized matching, visible-only uniqueness, 0 -> next rung, >1 -> hard failure); `version` is the artifact revision.

Covered in detail in §5.

§3.3 — Deterministic replay

`replay.py` (1009). No model, and nothing it imports touches one. Covered in detail in §6 and §7.

§3.4 — Safety & policy

There is deliberately no `policy.py`. Guardrails are enforced at the layer that can actually enforce them:

- **Network allowlist at the network layer** (`surface.py:186`). Context-wide `page.route("**/*")`, so it covers clicks, redirects, popups, iframes and subresources — not just navigations the engine initiates itself. Defaults to exactly the artifact's entry host. Off-allowlist traffic that breaks the flow surfaces as `POLICY_VIOLATION`, not `FAILURE` (`replay.py:206`).
- **Mutating-by-default risk.** `_RiskWatch` (`planner.py:212`) observes traffic across the action **and its aftermath**, because a click's POST commonly fires after the driver call returns. Any non-GET marks the step `risky`. Signals only ever raise risk, never lower it.
- **Hash-bound sign-off** (`artifact.py:502`). `risk_hash` covers all artifact content except the signature block, so any edit voids prior approval. Unattended replay of risky steps without a valid signature is refused before the browser launches (`replay.py:150`).
- **Redaction.** Sensitive params/outputs are «masked» in traces, results, and transcripts. A sensitive parameter must not carry an example value (`artifact.py:349`); its value comes from `HANDS_PARAM_<NAME>`, never a request file (`discover.py:140`).

- **Redaction canary.** After a human has driven the session, page-content evidence is suppressed wholesale (`replay.py:218`) — a screenshot would persist what they typed.

Prompt-injection posture: page text is fenced as untrusted in the observation (`observe.py:87`) and the system prompt says to read it as data. This is weak by construction and worth saying so. The real defense is that the model can only act on refs and cannot reach the network.

§3.5 — Evidence / observability

`trace.py` — append-only JSONL, one directory per run under `runs/`, flushed per event. There is an event for every **decision**, not just every action:

```
run_started · requires_met/unmet · step_started · fragile_target_warning
recognizers_suppressed_stale · recognizer_suppression_lifted
acted · act_attempt_failed · action_effect_detected · postconditions_met
recognizer_fired · recovery_step_started/done · checkpoint_verified
output_extracted · escalation_raised · control_granted · human_action
human_command_executed · resume_decision · evidence_suppressed_human_window
policy_blocked_requests · escalation_ttl_expired · run_finished
```

`run_started` records the **sha256 of the exact artifact that ran**, so every trace line is attributable to one reviewable contract.

Richer signal on failure: full-page screenshot **plus per-frame accessibility snapshot** (`surface.py:369`), best-effort so evidence capture can never turn a reportable failure into a crash.

§3.6 — Escalation & handoff

`escalation.py` (327) + engine glue in `replay.py`. Covered in detail in §8.

§3.7 — Heterogeneity & multi-tenant (design)

The seam is **one module wide**: `surface.py` is the only file that imports Playwright. Everything above it speaks artifact vocabulary (ladders, context paths, conditions, regions).

Concretely surface-neutral in the schema:

- `Entry` is a typed union with a `web` variant (`artifact.py:137`) — a desktop surface adds its own variant (app id + window title).
- `ContextSegment` is documented as “one segment of a surface context path (a frame, **on the web surface**)” — it maps to a window/pane elsewhere.

- `container_of` is documented as “nearest ancestor table on web, the enclosing group/pane on desktop” (`surface.py:301`).
- The CSS rung is explicitly web-only and last-resort, and carries a `fragile` flag; cross-tenant replay would disable it entirely.

Built toward multi-tenant: per-step rung telemetry in the trace (which rung actually carried each step), the `fragile` flag promoted into the **reviewable contract** rather than buried in a log, `scrub_url_pattern` canonicalizing `/member/12345` -> `/member/[^/]+` (`recorder.py:319`), and a drift eval — renamed label produces a loud failure listing every rung tried, in 0.5s.

Not built: overlays/variants, drift-telemetry aggregation, a second tenant. That is the honest line, and §11 states it plainly.

5 • The artifact, annotated

The shipped example, `capabilities/lookup_member_balance.json`:

```
parameters: member_id (string, pattern [0-9A-Za-z]{1,10}, example "12345")
outcomes:    MEMBER_NOT_FOUND · VALIDATION_REJECTED · PERMISSION_DENIED
outputs:     savings_balance (decimal, sensitive, parse money/en_US,
                        region "accounts", from: cell_right of text "Savings")
regions:     results · member_header · accounts · input_errors
steps:       s1 type "Enter the member ID"    safe ladder [label, relative]
             s2 click "Submit the search"      safe ladder [role]
conditions:  not_found          business_outcome MEMBER_NOT_FOUND   armed_after s2
             validation_rejected business_outcome VALIDATION_REJECTED armed_after s2
             permission_denied  business_outcome PERMISSION_DENIED  armed_after s2
             session_expired    recoverable                          armed_after s1
             maintenance_interstitial recoverable                    armed_after s1
checkpoint:  heading "Member Details" in frame "main"
             AND region "member_header" text matches param member_id
```

Note the parameter pattern is `[0-9A-Za-z]{1,10}` while the fixture accepts only `[0-9]{1,10}`. That is deliberate: the artifact lets an alphanumeric value through so the app's own server-side validation can be exercised and classified as `VALIDATION_REJECTED`. Client-side pattern checking that pre-empted the app would hide a whole class of runtime state.

The load-time integrity validators

All in `Capability._integrity` (`artifact.py:412`). These are where the judgment lives, and each one exists because of a specific failure mode.

Outcome closure, both directions (`:418`). Every business-outcome recognizer maps to a declared code, **and** every declared code has a recognizer. Forward closure stops the artifact returning a code the caller never heard of; reverse closure stops the contract advertising an outcome that can never fire. The caller's contract and the artifact's behavior cannot drift apart.

Checkpoint identity binding (`:492`). A capability with parameters must prove it reached the record it was *asked about* — via a `region_text_matches_param` condition — or explicitly set `allow_unbound_checkpoint` with justification. This is the single most important validator in the file. “A details page loaded” passing while the *previous* member is still on screen is the silent-wrong-answer bug, and it is the failure mode that would actually hurt a bank.

Param-used (:475). Every declared parameter must be referenced by a typed placeholder, a value postcondition, or an identity binding. An unused parameter is a contract lying about its inputs.

Condition class coherence (:277). A business outcome may not carry recovery steps — “an outcome is an answer, not a recovery”. A recoverable condition must carry recovery steps *and* an explicit resume point, and `restart_from` must be set if and only if `resume == "restart_from"`.

post: list[PostCondition] = Field(min_length=1) (:326). An action with no postcondition **cannot be represented in the schema**. There is no such thing as an unverified step in this system.

Region references resolve · step ids unique · armed_after and restart_from point at real steps · sensitive params carry no example (:349) · **decimal outputs require a money parse spec** (:379).

Why this shape and not a step list

Three reasons worth stating:

1. **A calling agent needs to invoke it blind.** If `parameters`, `outputs`, and `outcomes` are not at the top level, the caller has to read the step list to work out what the capability needs and what it can return. That makes every consumer depend on implementation detail.
2. **A human reviewer needs to sign it.** The risk review binds to a hash of the whole artifact, so the reviewable unit and the executable unit are the same document. A diffable step list is what makes review possible at all.
3. **The closed outcome set is the contract's teeth.** `outcomes` being a declared dict, closed in both directions, is what lets a caller write an exhaustive handler. That is impossible against a system that can return arbitrary error strings.

6 · Determinism

The locator ladder

`surface.py:249` . Semantics pinned in `schema_version` .

```
role + accessible name    <- what a human operator sees
label (label[for], aria-label only)
exact visible text
geometric relative       <- how you read table-soup
CSS path                 <- web-only, last resort, sets fragile flag
```

The rule that matters:

0 matches -> try the next rung. More than 1 match -> hard failure. Lower rungs are fallbacks, **not** disambiguators.

Descending the ladder on ambiguity would be guessing, and guessing is invariant #1. This is the single most-defensible line in the codebase.

Supporting details:

- Matching is **exact and whitespace-normalized**.
- Uniqueness is counted over **visible elements only** (`surface.py:396`). Text and CSS strategies match hidden elements — collapsed panels, duplicated mobile/desktop markup — and counting those makes the uniqueness rule meaningless.
- `_innermost` (`:415`) handles nested exact-text matches: when an element and its ancestor both match, keep only elements that do not contain another match.
- Every target carries a **fingerprint** (role, editable) verified on resolution (`:339`). *Uniqueness is not correctness* — the fingerprint is the check that a unique match is also the right match.
- Label and proximity are **separate rungs** on purpose: `label` is real label association only, proximity lives in `relative` . Telemetry then shows which mechanism actually carried the step, which is the drift signal that matters.

Geometric rungs — reading table-soup

`_relative_matches` (`surface.py:297`). Three relations: `cell_right` , `nearest_input_right` , `container_of` .

Two decisions to defend:

Same-row means the boxes actually overlap vertically (`:324`):

```
overlap = min(anchor.bottom, box.bottom) - max(anchor.y, box.y)
same_row = overlap >= 0.5 * min(anchor.height, box.height)
```

A center-distance tolerance would admit the adjacent row — for instance a table’s header cell in the same column. Overlap is the geometric statement of “same visual row.”

A geometric tie is ambiguity (`:334`):

```
best = min(distance for distance, _ in scored)
return [c for distance, c in scored if distance - best <= 0.75]
```

Returning the DOM-order winner would smuggle a guess past the exactly-one rule (a rowspan anchor sees one equally-near cell per spanned row). All tied candidates are returned so `resolve` refuses.

The timeout model — two levels

There is no `sleep` -as-synchronization anywhere in the engine.

- Playwright’s own action auto-waiting gets a **short per-attempt timeout** (2000ms default).
- The engine owns a **per-step budget** (10s default) and runs `act` as a bounded retry loop, checking recognizers *before* the first attempt and *between* attempts (`replay.py:369`).

The payoff: a known interstitial that appears between steps gets **recognized and classified** rather than smashed into one long opaque `TimeoutError` . A single long timeout cannot distinguish “slow” from “session expired” from “the button is gone”, and those three need three different responses.

The freshness rule — two halves

Business outcomes only. `replay.py:343` and `replay.py:617` .

Half one — the snapshot. A recognizer already matching *before* a step's action is suppressed for that step. Stale “No member matches” left over from a previous search must not answer the current one. Without this, the engine would confidently report `MEMBER_NOT_FOUND` for a member that exists.

Half two — suppression lifts. Once the stale state is observed *gone*, the recognizer is live again (`_refresh_suppression`). Without this, a genuine new match of the same condition within the same step would be invisible — the first half's fix would create a second bug.

Recoverable conditions are never suppressed. A blocking state present before the action is exactly what recovery exists for (`conditions.py:127`).

Precedence: a satisfied postcondition beats a simultaneously-matching recognizer (`replay.py:463`). If the step demonstrably worked, it worked.

No double fire

The one failure this engine must make structurally impossible.

- **Risky steps are never auto-retried** (`replay.py:386`). Retry requires `step.risk == "safe"`.
- Before **any** re-act, `_effect_already_present` (`:642`) probes whether the action already landed. A click that timed out client-side may have succeeded server-side. It re-acts only if the effect is provably absent, and it declines to guess when the postcondition needs a resolved target it does not have.
- A recoverable condition interrupting a risky step **post-action** is a hard failure, not a recovery (`:552`) — recovery cannot make a double-fire safe, but a human can.
- Handback resumes via a **forward position scan** (`:825`) and **refuses to re-run a risky step blind**.

Values are parsed, never string-compared

`values.py`.

`parse_money` (`:58`) validates strict en_US structure **before** stripping commas:

```
_US_MONEY_RE = re.compile(r"^(?:\d{1,3}(?:,\d{3})+|\d+)(?:\.\d+)?$")
```

So "1.234,50" — a comma-decimal locale rendering — **fails** rather than silently parsing to a 100x-wrong number. It also rejects NaN and Infinity, which a bare Decimal() would happily accept. Accepts \$1,234.50, (1,234.50) parenthesized negative, 1,234.50- trailing minus, leading minus, stray whitespace and nbsp (legacy pages pad with). Locale is declared in the artifact's parse spec, never guessed. **A parse failure is a FAILURE, never a value.**

value_bound_in (:87) makes identity binding boundary-anchored, not bare substring containment: member 123 must **not** match a page showing 12345 .

param_value_matches (:98) is type-aware because UIs reformat what you type — and an expected value that normalizes to nothing can never match, or a digitless param against an empty field would be vacuously true.

Measured results

From evals/results.md , regenerated offline by evals/run_evals.py :

SCENARIO	EXPECTED	CORRECT	LATENCY
happy path	success	yes	1.4s
fresh params	success	yes	0.8s
member not found	business_outcome:MEMBER_NOT_FOUND	yes	0.8s
server-side validation	business_outcome:VALIDATION_REJECTED	yes	0.8s
permission denied	business_outcome:PERMISSION_DENIED	yes	1.1s
session expired	success (recovered)	yes	1.4s
maintenance interstitial	success (recovered)	yes	1.4s
transient slow load (5s)	success	yes	5.7s
app error (HTTP 500)	failure	yes	10.8s
UI drift (renamed label)	failure	yes	0.5s

10/10 classified correctly. 5/5 recognizers exercised.

Stability: 10 runs, 10/10 success, mean 0.85s, max 1.13s.

Economics: discovery 3 model calls / 3,462 tokens / 3.87s. Replay **0 model calls, 0 tokens** (proven hermetically), 0.85s.

The zero-LLM proof

`tests/test_zero_llm.py` + `tests/hermetic_runner.py` . Three independent mechanisms, because an in-process import check would be order-dependent and name-brittle:

1. **Fresh subprocess** with every API-key-shaped environment variable stripped (regex `KEY|TOKEN|SECRET|ANTHROPIC|OPENAI|CREDENTIAL`).
2. **Socket guard**: `socket.socket.connect` is monkey-patched to raise on any non-loopback address.
3. **Import assertion** after the run: `openai` , `anthropic` , `hands.llm` , `hands.planner` , `hands.discover` must not be in `sys.modules` .

Plus a trace assertion that every emitted event is a known engine event.

`cli._discover` imports discovery lazily precisely to keep #3 true.

7 · Error taxonomy

The five-way result contract

`results.py`.

RESULT	MEANING	CARRIES	EXIT
SUCCESS	the flow completed	declared outputs	0
BUSINESS_OUTCOME	a legitimate answer	code, description, MatchEvidence	0
FAILURE	the flow broke	step, intent, expected, observed, screenshot, ally snapshot	1
PRECONDITION_FAILED	the environment is wrong	unmet condition, observed	1
POLICY_VIOLATION	guardrails refused	rule, detail	1

Three decisions to defend:

BUSINESS_OUTCOME exits 0 (`cli.py:93`). An answer is not a malfunction. “No such member” is the correct, useful response to a lookup for a member who does not exist, and a caller that treats it as an error will build retry logic around a fact.

BUSINESS_OUTCOME carries **MatchEvidence** — condition id, region, role, and the actual matched text. The result is auditable rather than a bare claim. When someone asks “how do you know it was not found?”, the answer is a quoted line from a named region, not “the code said so.”

PRECONDITION_FAILED is separate from **FAILURE**. If the session landed on a login page, the honest report is “the environment is wrong”, not a locator error on step 1. Conflating them sends the debugger to the wrong place.

POLICY_VIOLATION is separate too. The flow did not break; policy refused it. Different owner, different fix.

Three classes of runtime state

The brief asks for expected outcomes vs recoverable conditions vs hard failures. The schema makes this a closed choice: `Condition.classify` is `business_outcome | recoverable`, and anything unrecognized becomes **FAILURE**.

Business outcome — ends the run with an answer. Freshness rule applies. Requires an `outcome_code` ; may not carry recovery steps.

Recoverable — a blocking state handled in place. Never suppressed. Requires recovery steps and an explicit resume point, bounded by `max_fires_per_run` (default 2).

The resume point matters: plain “continue” is wrong whenever recovery navigates. `resume` is `retry_current_step` | `restart_from`, and `restart_from` names a step. A session-expired recovery lands you back where you were going, which may be mid-flow.

Recovery is one level deep, never nested (`replay.py:571`). Recovery steps are single-attempt, with bounded postcondition waits and **no recognizer matching**. Any problem inside recovery promotes to a hard `FAILURE` that names the original condition. This deliberately avoids a fourth, unnamed state: “recovery is itself broken.”

Fire caps are loud on exhaustion (`:543`). A condition that fires more than `max_fires_per_run` times raises a failure naming the cap. An interstitial that reappears forever is a bug, not something to grind against.

Recognizer provenance and arming

- `provenance: discovered | authored` — where the recognizer came from.
- `verified_by_eval: bool` — whether an eval exercised it. **See §12: this is currently false everywhere.**
- `armed_after: <step id>` — recognizers are armed from that step’s action onward, not from run start. An unarmed recognizer cannot fire on a page it was never meant to classify.

A happy run never sees a not-found page, so each business outcome gets its **own discovery run** (`discover.py:299`). `_arming_step` (`:365`) then computes the happy-run step the outcome run reached — and **refuses** if the outcome run is not a structural prefix of the happy path (same action kinds, same primary rung). A positional guess across divergent flows would arm the recognizer at the wrong step, so divergence is surfaced rather than guessed around.

8 · Escalation & handoff

`escalation.py` (327) + `replay.py` glue. This is the part to lead with, because a real constraint produced the design rather than a convenient story.

The control-transfer model

- The control token is **engine-owned in-memory state** (`EscalationHub`). All mutation happens under one lock.
- The console runs on a **daemon HTTP thread in the same process and never touches Playwright** — the sync API is thread-affine. It only posts transition **requests** into the hub.
- The engine polls the hub at every tick of loops it **already has** (`replay.py:492`) and parks on a pending pause. The console shows control as granted only **after** that acknowledgment (`:760`).

Two drivers on one session is **structurally impossible**, not discouraged.

That sentence is the whole design. It is also why the sync API was the right choice: an async design would have made the wrong thing easy.

The state machine

```
AUTOMATION --pause--> PAUSED --> { HUMAN | AUTOMATION (approve) | FAILED (TTL) }
HUMAN --> { AUTOMATION (handback) | ABORTED (reason) | RESOLVED (code, note) }
```

Five operator decisions:

DECISION	MEANING
<code>request_control</code>	operator-initiated: stop it when you see it going wrong, at poll-tick granularity
<code>APPROVE</code>	resume automation without taking over
<code>HANDBACK</code>	the human is done; resume via forward scan
<code>ABORT</code>	the operator judges the run unsafe; records a reason
<code>RESOLVE</code>	the human established a business outcome manually, by declared code

Every transition records the operator identity.

RESOLVE is worth calling out: a human can end the run as a declared business outcome they established by hand, and the engine validates the code against `capability.outcomes` (`replay.py:770`) — an invalid code keeps them parked rather than escaping the closed outcome set.

Taking control of the *same* live session

Two paths through identical token machinery:

Headed. The operator drives the visible browser directly. Their actions are captured by listeners injected at **context creation** (`surface.py:148`) — not at takeover time, because listeners injected then would die on the first navigation.

Headless. The console offers a minimal click/type-by-role command channel, and the **parked engine thread** executes each command on the operator's behalf (`replay.py:786`). Same token machinery, no thread-affinity violation, and the handoff becomes testable in CI.

Captured human actions are **semantic only** — never field values, just a masked length. The reasoning in the code comment: on legacy surfaces SSNs live in plain text inputs and no heuristic can find them all, so values are **categorically excluded** rather than filtered.

Bounded, not open-ended

- **TTL expiry** (`replay.py:774`): an unanswered intervention fails and the session closes. No keep-alive games with a banking session.
- **max_escalations = 3** per run, so escalation cannot ping-pong.
- **Context carried** into the intervention: capability, version, step id, intent, reason, masked params, expected state, remaining step intents, recent trace tail, screenshot path. Written to `intervention-N.json` in the run dir.
- **Resume is a forward position scan** (`:825`): checkpoint first — the human may have completed the whole flow — then the furthest step whose postconditions hold, else retry current. Unless the current step is risky, in which case it fails loudly rather than risk a double-fire.

Tested in `tests/test_escalation.py` (251 lines). Real handoff evidence in `evidence/escalation-run/`.

What is mocked, stated plainly

The operator console is a mock: a stdlib HTTP server with a 2-second meta-refresh and form-post controls. There is no co-browsing, no WebRTC, no real-time video. **The handoff mechanism and the control-transfer model are real**; the operator's window onto the session is a screenshot plus a command form.

9 · The discovery seam

Three ideas, and this is the part most likely to be probed as “how do you know the model didn’t just get lucky?”

Contract-first

`DiscoveryRequest` (`discover.py:66`) declares parameters, outputs, outcome codes, and **known example values** up front. The model only learns **how**.

It never invents the contract, which means the contract cannot be wrong in a way that only shows up at invocation time. Coherence is validated (`discover.py:80`): example params must cover exactly the non-sensitive parameters, `identity_param` must be declared, and each outcome’s params must cover the same set. Sensitive values come from `HANDS_PARAM_<NAME>`, never a request file.

Model proposes, recorder proves

`recorder.py`. Every semantic the model proposes is verified mechanically against the live page before it may enter the artifact.

`build_ladder` (`:81`). Candidate rungs in semantic-first order. Each is round-tripped through **the same locator engine replay uses**, and compared against the **pinned handle** from observation time:

```
same = bool(resolved.locator.evaluate("(a, b) => a === b", node.handle))
```

A rung that does not resolve uniquely to the acted element is **dropped**, not recorded as a latent replay bug. If only a CSS path survives, the target is flagged `fragile` — in the artifact, not in a log.

`_reject_param_text` (`:141`). **Data-independence**: no distilled anchor, heading, marker, or URL pattern may contain this run’s parameter value. Otherwise it works for one invocation and no other. Applied to checkpoint headings, identity anchors, output anchors, outcome markers, and region anchors. `scrub_url_pattern` (`:319`) does the same for URLs and raises if a value survives scrubbing.

`build_identity_checkpoint` (:177). Proves the heading is *actually visible right now*, the identity region resolves, and it contains the invocation's parameter value with proper boundaries. A model-proposed heading that is not visible never enters the checkpoint.

`build_output` (:219) — **supervised**. The anchor must be data-independent, resolve next to a value, the value must parse, **and** it must match the expected value declared in the request. The comment says it best: a wrong anchor that happens to sit next to money must not become a capability that returns wrong balances forever.

`build_outcome_condition` (:283). The region must resolve and the marker must be present in it right now, scoped structurally — never bare text anywhere on the page, or help copy reading “if no member matches...” would trigger a business outcome.

`_settle_post` (`planner.py:519`). Postconditions are derived from **what actually changed**, in preference order: the typed value itself, a newly appeared heading attributed to the frame it appeared in, then the acted frame's URL change with parameter values scrubbed. A step that changed nothing observable raises `DistillationError` — **never a step with an invented postcondition**.

`assemble` (:350) runs the full `Capability` validation. A validation failure here is a distillation error: the run happened, but its record is not trustworthy.

One run per ending

A happy run never sees a not-found page. So discovery runs the happy path once for steps, checkpoint and outputs, then **one run per declared outcome** for a live-verified recognizer each (`discover.py:185`). Recognizers cannot be authored from imagination and marked discovered.

`_entry_requires` (:395) records the entry-state contract — the first visible heading on the entry page — so replay reports `PRECONDITION_FAILED` rather than a confusing locator error when the session lands somewhere unexpected.

10 · Evidence inventory

evidence/	
README.md	what each run shows
artifact.json	the signed artifact
discovery-report.json	token/latency/ending summary
discovery-run/	trace.jsonl + transcript.json (happy leg, real LLM)
discovery-run-outcome/	trace.jsonl + transcript.json (outcome leg, real LLM)
replay-run/	trace.jsonl (success)
replay-run-not-found/	trace.jsonl (business outcome)
replay-run-validation/	trace.jsonl (business outcome)
replay-run-recovered/	trace.jsonl (recoverable -> success)
escalation-run/	trace.jsonl, intervention-1.json, failure.png, failure-a11y.txt (live handoff)

The model transcript lives in `transcript.json`, **decoupled from the artifact** — the artifact contains none of it. That is the §3.3 requirement that the artifact be independent of the raw model transcript.

11 · Built vs designed-not-built

Being crisp about this line is worth more than pretending the line is elsewhere.

Built and tested

Replay engine with the full five-way taxonomy · discovery (planner + recorder, real LLM evidence) · locator ladder including geometric rungs and fingerprints · recognizers with provenance and arming · identity-bound checkpoints · supervised typed outputs · network-layer allowlist · mutating-by-default risk with hash-bound sign-off · escalation state machine, operator console, resume scan, TTL, redaction canary · hermetic zero-LLM proof · 10-scenario eval table.

Designed, not built

GAP	WHAT EXISTS INSTEAD
Multi-tenant overlays / variants / drift-telemetry aggregation	per-step rung telemetry, the <code>fragile</code> flag, a drift eval (renamed label -> loud failure)
Desktop surface	schema is surface-neutral; only <code>WebSurface</code> exists
Native JS dialog (<code>alert</code> / <code>confirm</code>)	no <code>page.on("dialog")</code> exists — DESIGN describes it, it is not built
<code>{secret:ENV_VAR}</code> token	only <code>{param:name}</code> ; sensitive values use the <code>sensitive</code> flag + <code>HANDS_PARAM_<NAME></code>
Element-level taint masking	evidence is suppressed wholesale during the human window
Per-step <code>approve:</code> at invocation	the gate is signed-review vs attended, no per-step flag
Discovery-side escalation	replay-side is fully wired; a stuck planner ends the run

Cut legs: saucedemo portability, a Lakeside second tenant.

12 · Known discrepancies

Found on a fresh read-through. A reviewer can find these, so name them first.

1. `verified_by_eval` is `false` on every condition in the shipped artifact — even though `evals/results.md` shows all five recognizers exercised and passing. The field exists and nothing sets it. This is the one worth fixing: it converts “we have a provenance field” into “the eval run stamps it,” which is a much better answer to *“how do you know your recognizers work?”*
 2. `Observation.headings()` (`observe.py:108`) is dead code. `planner._headings_by_frame` superseded it and is frame-attributed, which is the correct version — a heading postcondition must be attributed to the frame it actually lives in, or it is false on the very page it was distilled from.
 3. `FALLBACK_MODEL` (`llm.py:22`) is defined and never referenced.
 4. `CLAUDE.md` describes “policy” as a shared module; there is no `policy.py`. Not wrong in substance — enforcement is split across `PolicySettings` in `replay.py`, the route handler in `surface.py`, and `risk_hash` in `artifact.py` — but the doc implies a file that does not exist.
-

13 · Likely hard questions

“Your replay only has two steps. Is this real?” The step count is a property of the fixture flow, not the engine. What is exercised is every mechanism: two ladder kinds including a geometric rung, an identity-bound checkpoint, a supervised money output, three business-outcome recognizers, two recoverable conditions with different resume points, and ten runtime-state scenarios. A ten-step flow would add steps, not mechanisms. The honest weakness is that no *risky* step appears in the shipped artifact, so the mutating path is covered by tests rather than by the demo flow.

“What happens when the UI actually changes?” It fails loudly and tells you every rung it tried, in 0.5s — that is the drift eval. What it does **not** do is silently resolve to the wrong element, because the CSS rung is last, flagged `fragile`, and every resolution is fingerprint-verified. The thing I have not built is aggregating that failure across tenants into a drift signal.

“Why not just let the model fix it at replay time?” Because then it is not deterministic, and the cost and audit story collapse. That said, a bounded single-step LLM recovery under policy is the stretch goal I would reach for next, recorded as evidence — never open-ended.

“How do you know the recorded locators actually work?” Because they were round-tripped through the real engine at record time and dropped if they did not resolve uniquely to the acted element. Not “the model said this selector is good.”

“What stops a wrong answer?” Four layers: fingerprint verification (a unique match must also be the right match), identity-bound checkpoints (the right *record*, not just the right *page*), strict money parsing (reject rather than mis-parse), and boundary- anchored identity (`123` does not match `12345`). Invariant #2 is the one I designed hardest against.

“Your operator console is a mock.” Yes, deliberately, and it is documented as such. The mock is the *window* — the mechanism is real: engine-owned token, structural single ownership, the same live session, captured human actions, TTL, forward-scan resume with a risky-step refusal. A co-browsing UI is a product surface; the control-transfer model is the engineering problem.

“Prompt injection?” Weak by construction, and I would rather say so than oversell it. Page text is fenced as untrusted and the prompt says to treat it as data, but that is a soft defense. The hard defenses are structural: the model can only act on refs from the current observation, cannot type a concrete parameter value, and the browser context cannot reach a host outside the allowlist.

“Why is a business outcome exit code 0?” Because an answer is not a malfunction. A caller that treats “no such member” as a failure will build retry logic around a fact. This is the single most common design mistake the brief calls out, and making it structural — a separate result shape carrying evidence, in a closed declared set — is the response.

“What would you build next?” In order: stamp `verified_by_eval` from the eval run; native dialog handling (the brief names unexpected confirmation dialogs, and DESIGN describes a mechanism I did not build); then cross-tenant variants with per-tenant overrides, which is the largest designed-not-built item and the one that matters most at hundreds-of-tenants scale.

14 · Commands

Python $\geq$ 3.12. Editable install with dev extras. Only discovery needs a key.

```
python3 -m venv .venv
.venv/bin/pip install -e ".[dev]"
.venv/bin/playwright install chromium
```

Checks — all three must be clean before claiming anything is done:

```
.venv/bin/pytest          # offline, ~50s, spins the fixture itself
.venv/bin/ruff check .
.venv/bin/mypy            # strict
.venv/bin/python evals/run_evals.py # regenerates evals/results.md, offline
```

Run the target app (terminal 1):

```
.venv/bin/python -m fixture.app --port 8000
```

Replay — no key, no network beyond localhost:

```
.venv/bin/hands replay capabilities/lookup_member_balance.json --param member_id=12345
.venv/bin/hands replay <artifact> --headed      # watch the browser
.venv/bin/hands replay <artifact> --attended    # failures -> console on :8321
```

Sign risk labels (binds to artifact hash; any edit voids it):

```
.venv/bin/hands review <artifact> --operator <name>
```

Discovery (needs `GROQ_API_KEY` in `.env` ; hits the model):

```
.venv/bin/hands discover capabilities/requests/lookup_member_balance.request.json
```

Inject a fault into the fixture:

```
curl -X POST localhost:8000/__faults -H 'Content-Type: application/json' \
-d '{"fault": "session_expired", "enabled": true}'
```

Faults available: `slow_load`, `permission_denied`, `session_expired`, `interstitial`, `app_error`, `renamed_label`.

Editing gotchas

- **Never import from `hands.llm`** (or anything model-touching) inside the replay path. The hermetic test asserts no model modules load; `cli._discover` imports discovery lazily on purpose.
- Frame handles are re-walked from the page root every poll (`frame_for`) — never cache a frame or locator across polls; legacy pages reload frames.
- New artifact fields go in `artifact.py` with a validator **and** a schema test in `tests/test_artifact_schema.py`; artifacts are round-tripped by `dump_capability` / `load_capability`.
- `replay.py` uses exceptions for control flow (`_Recognized` , `_StepFailed` , `_Recover` , `_PauseRequested`). Keep that shape; do not return sentinels through the step loop.
- All schema invariants live in `@model_validator`s in `artifact.py`, not in the engine.